

DSCI-552 HW 0

Introduction to Basic Development Tools

Name: Alyssa

GitHub Username: akeehan56

Email: akeehan@usc.edu

USC ID: 3429542433

Rules

1. Please read the instructions and problem prompts **carefully**.
2. This lab is to give you some basic APIs of numpy, pandas and scikit-learn. Besides, some topics such as how to make your jupyter notebook be a more efficient developing tools, how to use git and GitHub will also be covered. The lab is to be done individually. You may talk to your fellow classmates about general issues ("Remind me again: Which API should I used for doing group by operation to a data set") but about the specifics of how to do these exercises.
3. Along with a similar vein, you can ask the TA for help, but ask questions about **concepts** but not ask the TA to help you debug your code. The TA is here to help, but not to do the work for you.
4. You are welcome to use the class resources and the Internet.
5. Playing with variations. Solve one problems, and then copy the code to a new cell and play around with it. Doing this is the single most important thing when learning programming.
6. This lab will not be graded but the content is highly related to your future programming assignments. So, treat it wisely.
7. All the content having been gone though in the week 1 discussion is just a snapshot of the most basic concepts. **You need to keep study more about Git, GitHub, Pandas, Numpy and Scikit-Learn in order to finish your programming assignments successfully.**
8. Have fun!

Setup Development Environment

There are many ways to setup the environment. But, I do recommend a simple idea that is using the Anaconda, which is a pre-build python environment with bundles of useful packages.

To download the Anaconda, go to the following website:

<https://www.anaconda.com/distribution/>. Download the correct version based on your operating system and install it step by step.

Then, **configure your PATH environment variable** to make the conda command work. The following command is an easy way to test whether your configuration is correct. If it is, you will see something as like as the sample output.

command:

```
conda --version
```

sample output:

```
conda 4.6.12
```

Finally, download this jupyter notebook file, then change the working directory to where its location in terminal, and type the following command to open the jupyter notebook and finish the lab.

command: jupyter notebook

```
In [60]: import pandas as pd
import numpy as np
```

Pandas

The read_csv() Method

First, read the documentation about the `read_csv()` method in Pandas (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html).

Then, try to read data from file Salaries.csv to a dataframe, make the column playerId in the csv file as the index column and the first row as the header. Also, skip the second row when reading the file.

```
In [61]: salaries = pd.read_csv('Salaries.csv', header = 0, skiprows=[1], index_col='
salaries
```

Out [61]:

	yearID	teamID	lgID	salary
playerID				
bedrost01	1985	ATL	NL	550000
benedbr01	1985	ATL	NL	545000
campri01	1985	ATL	NL	633333
ceronri01	1985	ATL	NL	625000
chambch01	1985	ATL	NL	800000
...	...	...	...	...
treinbl01	2015	WAS	NL	512800
ugglada01	2015	WAS	NL	507500
werthja01	2015	WAS	NL	21000000
zimmejo02	2015	WAS	NL	16500000
zimmery01	2015	WAS	NL	14000000

25574 rows × 4 columns

Indexing and Selecting Data

Select the id of the players who are registered in ATL and HOU and whose salary is higher than one million.

```
In [62]: atl_hou_salaries = salaries[salaries["teamID"].isin(["ATL", "HOU"]) & (salaries["salary"] > 1000000)]
atl_hou_salaries
```

```
Out [62]: Index(['hornebo01', 'murphda05', 'suttebr01', 'ryanno01', 'hornebo01', 'murphda05', 'suttebr01', 'ryanno01', 'griffke01', 'murphda05', 'feldmsc01', 'gonzama01', 'gregelu01', 'lowrije01', 'neshepa01', 'quallch01', 'rasmuco01', 'singljo02', 'sippto01', 'valbulu01'],
dtype='object', name='playerID', length=595)
```

The describe() Method

Calculate the standard Deviation, first quartile, medium, third quartile, mean, maximum, minimum of the salary in team ATL.

```
In [63]: atl_salaries = salaries[salaries.teamID=='ATL']
atl_salaries_std = atl_salaries["salary"].std()
atl_salaries_quantiles = atl_salaries["salary"].quantile([0.25,0.50,0.75])
atl_salaries_mean = atl_salaries["salary"].mean()
atl_salaries_max = atl_salaries["salary"].max()
atl_salaries_min = atl_salaries["salary"].min()
print('standard deviation: ', atl_salaries_std)
print(atl_salaries_quantiles)
```

```
print('mean: ', atl_salaries_mean)
print('max: ', atl_salaries_max)
print('min: ', atl_salaries_min)

atl_salaries.describe()
```

```
standard deviation: 3434319.6054416075
0.25      300000.0
0.50      600000.0
0.75     2400000.0
Name: salary, dtype: float64
mean: 2207749.190960452
max: 16061802
min: 60000
```

Out[63]:

	yearID	salary
count	885.000000	8.850000e+02
mean	1999.725424	2.207749e+06
std	8.766470	3.434320e+06
min	1985.000000	6.000000e+04
25%	1992.000000	3.000000e+05
50%	1999.000000	6.000000e+05
75%	2007.000000	2.400000e+06
max	2015.000000	1.606180e+07

The iterrows() Method

Create a Python dictionary object whose keys are the headers of the dataframe created in the read_csv() exercise and values are Python list objects that contain data corresponding to the headers. (Here, use the iterrows method to iterate each row of the dataframe and copy it to a dictionary. However, there is a easier way. Learn how the to_dict() method works by yourself later)

```
In [64]: salaries = pd.read_csv("Salaries.csv")
salaries_dict = salaries.to_dict(orient="records")
salaries_dict[:10] # only show the first 10
```

```
Out[64]: [{ 'yearID': 1985,
  'teamID': 'ATL',
  'lgID': 'NL',
  'playerID': 'barkele01',
  'salary': 870000},
{'yearID': 1985,
  'teamID': 'ATL',
  'lgID': 'NL',
  'playerID': 'bedrost01',
  'salary': 550000},
{'yearID': 1985,
  'teamID': 'ATL',
  'lgID': 'NL',
  'playerID': 'benedbr01',
  'salary': 545000},
{'yearID': 1985,
  'teamID': 'ATL',
  'lgID': 'NL',
  'playerID': 'campri01',
  'salary': 633333},
{'yearID': 1985,
  'teamID': 'ATL',
  'lgID': 'NL',
  'playerID': 'ceronri01',
  'salary': 625000},
{'yearID': 1985,
  'teamID': 'ATL',
  'lgID': 'NL',
  'playerID': 'chambch01',
  'salary': 800000},
{'yearID': 1985,
  'teamID': 'ATL',
  'lgID': 'NL',
  'playerID': 'dedmoje01',
  'salary': 150000},
{'yearID': 1985,
  'teamID': 'ATL',
  'lgID': 'NL',
  'playerID': 'forstte01',
  'salary': 483333},
{'yearID': 1985,
  'teamID': 'ATL',
  'lgID': 'NL',
  'playerID': 'garbege01',
  'salary': 772000},
{'yearID': 1985,
  'teamID': 'ATL',
  'lgID': 'NL',
  'playerID': 'harpete01',
  'salary': 250000}]
```

Create Dataframe Using the Constructor

Read the documentation: <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.html#pandas.DataFrame> and create a

dataframe using `pd.DataFrame` from the dictionary created in the `iterrows()` exercise. Change the header to "a", "b", "c", ... at creation time.

```
In [65]: df = pd.DataFrame(salaries_dict)
df.columns = ['a', 'b', 'c', 'd', 'e']
df
```

```
Out [65]:
```

	a	b	c	d	e
0	1985	ATL	NL	barkele01	870000
1	1985	ATL	NL	bedrost01	550000
2	1985	ATL	NL	benedbr01	545000
3	1985	ATL	NL	campri01	633333
4	1985	ATL	NL	ceronri01	625000
...	...	...	...	...	...
25570	2015	WAS	NL	treinbl01	512800
25571	2015	WAS	NL	ugglada01	507500
25572	2015	WAS	NL	werthja01	21000000
25573	2015	WAS	NL	zimmejo02	16500000
25574	2015	WAS	NL	zimmery01	14000000

25575 rows × 5 columns

Numpy

Quick start: <https://www.numpy.org/devdocs/user/quickstart.html>

Numpy axes explanation: <https://www.sharpsightlabs.com/blog/numpy-axes-explained/>

The `np.array` Method

Example 1:

```
ls = [1, 2, 3]
arr = np.array(ls)
```

Example 2:

```
>>> np.array([[1, 2], [3, 4]])
array([[1, 2],
       [3, 4]])
```

Now, create a 2-dimensional Python list object, then convert it to a Numpy array object.

```
In [66]: ls = [[1,2,3],[4,5,6]]
arr = np.array(ls)
arr
```

```
Out[66]: array([[1, 2, 3],
               [4, 5, 6]])
```

ndarray Objects' Attributes

Play with the **ndim**, **shape**, **size**, **dtype**, **itemsize** and **data** attribute.

Example:

```
>>> arr = np.array([[1, 2], [3, 4]])
>>> arr.ndim
2
```

```
In [67]: print(arr.ndim)
print(arr.shape)
print(arr.dtype)
print(arr.itemsize)
print(arr.size)
print(arr.data)
```

```
2
(2, 3)
int64
8
6
<memory at 0x11c319630>
```

Dimension of ndarray Objects

Play with the **reshape()** and **flatten()** method.

Example:

```
>>> arr = np.array([[1, 2], [3, 4]])
>>> arr.flatten()
array([1, 2, 3, 4])
```

```
In [68]: arr_flatten = arr.flatten()
arr_flatten()
```

```
Out[68]: array([1, 2, 3, 4, 5, 6])
```

The Slice Operation of ndarray Objects

Understand how the slice operation works for 1-D array and 2-D array.

Example:

```
>>> arr = np.array([[1, 2, 3], [3, 4, 6], [7, 8, 9]])
>>> arr[1:]
array([[3, 4, 6],
       [7, 8, 9]])
>>> arr[1:, 0:2]
array([[3, 4],
       [7, 8]])
```

```
In [69]: arr = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(arr[1:])
print(arr[:1])
print(arr[:,1:])
```

```
[[4 5 6]
 [7 8 9]]
[[1 2 3]]
[[1 2]]
```

The Calculation of ndarray Objects

Play with the **argmin()**, **argmax()**, **min()**, **max()**, **mean()**, **sum()**, **std()**, **dot()**, **square()**, **sqrt()**, **abs()**, **exp()**, **sign()**, **mod()** method.

Example:

```
>>> np.square(array)
array([[ 1,  4,  9],
       [ 9, 16, 36],
       [49, 64, 81]])
```

```
In [ ]:
```

Other Important Methods Inside Module Numpy

Play with the **arange()**, **ones()**, **zeros()**, **eye()**, **linspace()**, **concatenate()** method.

Example:

```
>>> np.eye(3)
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

```
In [ ]:
```

Scikit-Learn

The followings are packages (or methods) in Python (Scikit-Learn and Scipy) that will be frequently used in your programming assignment. So, please read carefully.

- Data Preprocessing (<https://scikit-learn.org/stable/modules/classes.html#module-sklearn.preprocessing>)
 - Standardization: StandardScaler
 - Normalization: MinMaxScaler
 - Quantifying Categorical Features: LabelEncoder, OneHotEncoder
 - Construct Train and Test Set: model_selection.train_test_split
- KNN: KNeighborsClassifier
- Linear Regression: LinearRegression
- Logistic Regression: LogisticRegression, LogisticRegressionCV
- Feature Selection / Model Selection
 - L1 Penalized Regression (Lasso Regression) with Cross-Validation: LassoCV
 - L2 Penalized Regression (Ridge Regression) with Cross-Validation: RidgeCV
 - Cross-Validation: StratifiedKFold, RepeatedKFold, LeaveOneOut, KFold, model_selection.cross_validate, model_selection.cross_val_predict, model_selection.cross_val_score
 - Model Metrics (<https://scikit-learn.org/stable/modules/classes.html#sklearn-metrics-metrics>): accuracy_score, auc, f1_score, hamming_loss, precision_score, recall_score, roc_auc_score
- Decision Tree: DecisionTreeClassifier, DecisionTreeRegressor
- Bootstrap, Ensemble Methods
 - Bootstrap: bootstrapped (<https://pypi.org/project/bootstrapped/>)
 - Bagging: RandomForestClassifier, RandomForestRegressor
 - Boosting: AdaBoostClassifier, AdaBoostRegressor
- Support Vector Machines (<https://scikit-learn.org/stable/modules/svm.html#svm>): LinearSVC, LinearSVR
- Multiclass and Multilabel Classification (<https://scikit-learn.org/stable/modules/classes.html#module-sklearn.multiclass>)
 - One-vs-one Multiclass Strategy: OneVsOneClassifier
 - One-vs-the-rest (OvR) multiclass/multilabel strategy / OneVsRestClassifier
- Unsupervised Learning
 - K-means Clustering: KMeans
 - Hierarchical Clustering: scipy.cluster.hierarchy (not scikit-learn)
- Semisupervised Learning (https://scikit-learn.org/stable/modules/label_propagation.html)

Matplotlib

Quick start: <https://matplotlib.org/3.1.1/tutorials/introductory/pyplot.html>

Exercises:

(a) Create two one dimensional arrays x and y and plot y vs x, add title, xlabel, ylabel, grid.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.linspace(-5, 5, num=20)
y = np.array([j ** 2 for j in x])
```

copy the code above to the following cell and add code for plotting the parabola.

In []:

What happens if the independent variable is not sorted before plotting? Try plotting directly using the following defined array.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.linspace(-5, 5, num=20)
np.random.shuffle(x)
y = np.array([j ** 2 for j in x])
```

In []:

(b) Create multiple arrays and plot them with different styles, add legends, add text/mathematical equations on the plot.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.linspace(-5, 5, num=20)
y1 = np.array([j for j in x])
y2 = np.array([j ** 2 for j in x])
y3 = np.array([j ** 3 for j in x])
```

copy the code above to the following cell and add code for plotting curve $(x, y1)$, $(x, y2)$ and $(x, y3)$.

In []:

(c) Create multiple arrays and plot them into one figure (**No multiple figure and no subplot is allowed in this question**).

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.linspace(-5, 5, num=20)
y1 = np.array([j for j in x])
y2 = np.array([j ** 2 for j in x])
y3 = np.array([j ** 3 for j in x])
```

copy the code above to the following cell and add code for plotting curve (x, y_1) , (x, y_2) and (x, y_3) .

In []:

(d) Create multiple subplots, play around with the figure size, figure title, and its font style and font size (**One curve is plotted in one subplot in this question**).

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.linspace(-5, 5, num=20)
y1 = np.array([j for j in x])
y2 = np.array([j ** 2 for j in x])
y3 = np.array([j ** 3 for j in x])
```

copy the code above to the following cell and add code for plotting curve (x, y_1) , (x, y_2) and (x, y_3) .

In []:

(e) Change the limits on x and y axes, **use logarithmic axes to plot**.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.linspace(-5, 5, num=20)
y = np.array([j ** 2 for j in x])
```

copy the code above to the following cell and add code for plotting the parabola.

In []:

Pandas's DataFrame.plot and Seaborn

Pandas's DataFrame.plot

Use the Salaries.csv again (You can use the dataframe object loaded from section 3.1).

(a) For team 'ATL', plot a scatter plot between feature yearID and salary.

In []:

(b) For year 1985, plot a bar chart to show the average salary for each team.

In []:

(c) For team 'ATL', plot a line chart to show how the annual average salary change by years.

In []:

Seaborn

(a) Append one more numeric feature to the data frame (can be generated randomly), then for team 'ATL', use the seaborn.pairplot to plot scatter plots among all numeric features in the data frame for team.

In []:

(b) For year 1985 and for each team, plot a boxplot to show how the salary distribute within a team.

In []:

(c) Read the official documentation (<https://seaborn.pydata.org/>) to understand how Implot, catplot, relplot, and jointplot works.

Jupyter Notebook

Jupyter Notebook Extensions

Extensions such as the code formatter, table of content is to make your development more efficient. To explore it, please refer to https://github.com/ipython-contrib/jupyter_contrib_nbextensions.

Jupyter Visual Debugger

The Pixie Debugger is a visual debugger for debugging on Jupyter Notebook. To explore it, please refer to <https://medium.com/codait/the-visual-python-debugger-for-jupyter-notebooks-youve-always-wanted-761713babc62>.

Git and GitHub

1. In the directory that where this jupyter notebook file locates in, init a Git repository.
2. Checkout a new branch called dev and commit the current notebook within this branch.
3. Merge the dev branch to the master branch (the default branch).
4. Create a temporary repository (just for practicing and you can delete it later) in GitHub.

5. Push new changes in the master branch to the remote repository created in step 4.
6. Checkout the dev branch again and do some changes to your notebook, and then repeat step 3 and step 5.

In []:

In []:

In []:

In []:

In []: